AF 52

Synopsis

AI-Powered Incident Intelligence Platform for Automated Log Anomaly Detection and Root-Cause Explanation (AIOps)

Submitted in partial fulfilment of the requirements of the degree of

Bachelor of Engineering

In

Electronics and Computer Engineering

by

Mohit Chogale EU1237040

Ayush Mishra EU1237033

Under the Guidance of

Dr. Pandharinath Ghonge

Associated Professor, Department of Electronics & Computer Engineering, SJCEM



DEPARTMENT OF COMPUTER ENGINEERING

ST. JOHN COLLEGE OF ENGINEERING & MANAGEMENT

UNIVERSITY OF MUMBAI

2026-2027

St. John College of Engineering and Management
Autonomous Institute
(A Christian Religious Minority Institution)
Approved by AICTE and DTE, Affiliated to University of Mumbai / MSBTE
DTE Code : 3218 AICTE Permanent ID : 1-4790201
NAAC Accredited with Grade 'A+', Three Programs NBA Accredited

Certificate of Approval

This is to certify that **[Name(s) of Student(s)]**, are bonafide students of B.E. in Computer Engineering Department, St. John College of Engineering and Management, Mumbai. They have satisfactorily completed the requirements of Synopsis as prescribed by **ST. JOHN COLLEGE OF ENGINEERING AND MANAGEMENT (An Autonomous College affiliated to University of Mumbai),** while working on "**AI-Powered Incident Intelligence Platform for Automated Log Anomaly Detection and Root-Cause Explanation (AIOps)**".

Signature : .....
Name : **Dr. Pandharinath Ghonge**
Designation : Associated Professor

Signature : .....
Name : **Dr. Md. Imaduddin**
Designation : Head of Department

Signature : .....
Name : **Dr. Kamal Shah**
Principal,
St. John College of Engineering and Management.

Date: Place:

Declaration

I/we declare that this written submission represents my/our ideas in my/our own words and where others ideas or words have been included, I/we have adequately cited and referenced the original sources. I/we also declare that I/we have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in my/our submission. I/we understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

Mohit Chogale EU1237040

Ayush Mishra EU1237033

Date:

Acknowledgement

We express our sincere gratitude to all those who have directly and indirectly contributed to the successful preparation of this project synopsis.

We are deeply grateful to **Dr. Kamal Shah, Principal, St. John College of Engineering and Management**, for providing an excellent academic environment and the necessary facilities to undertake this project.

We extend our heartfelt thanks to **Dr. Md. Imaduddin**, **Head of the Department of Electronics & Computer Engineering** for their constant encouragement, valuable suggestions, and continuous support throughout the preparation of this synopsis.

We would like to express our sincere appreciation to our project guide, **Dr. Pandharinath Ghonge**, for their invaluable guidance, constructive suggestions, technical expertise, and continuous motivation. Their encouragement and insightful feedback have been instrumental in shaping this project synopsis.

We also acknowledge all the faculty members and laboratory staff of the Department of Computer Engineering for their timely assistance, cooperation, and technical support. We are also thankful to the open-source communities behind FastAPI, PostgreSQL, pgvector, Redis, scikit-learn, Ollama and Docker, whose freely available tools made this project possible.

Finally, we express our heartfelt gratitude to our parents, family members, friends, and classmates for their constant encouragement, moral support, and cooperation throughout this work.

Group Members

1. Mohit Chogale EU1237040
2. Ayush Mishra EU1237033

Abstract

Modern production software systems emit enormous volumes of logs and alerts, yet most monitoring tools only signal that a threshold has been crossed and leave engineers to work out what happened, why, and whether it has happened before. This manual triage is slow and depends heavily on the tribal knowledge of experienced engineers. This project presents an AI-Powered Incident Intelligence Platform that automates the first step of incident triage in Site Reliability Engineering (SRE). Logs from a synthetic generator or an external source, received through an API-key-secured REST endpoint, are buffered in a Redis Stream and persisted in PostgreSQL. Two complementary detectors then run in parallel: an explainable rule-based detector augmented with Drain3 log template mining, and an unsupervised Isolation Forest model trained on a rolling window of recent logs. Each anomaly is embedded with the all-MiniLM-L6-v2 sentence-transformer and matched by cosine similarity against a pgvector-backed library of past incidents, following a Retrieval-Augmented Generation (RAG) approach. The retrieved incident and the new anomaly are passed to a locally hosted Llama 3.2 model served by Ollama, which produces a plain-English explanation. A rules engine assigns severity and pushes high-priority alerts to Slack, while a human feedback loop writes operator corrections back into the vector store so that the system's memory improves over time. A FastAPI backend and a React dashboard expose the results, and the complete multi-service system is deployed with a single Docker Compose command. The system is evaluated using precision, recall and F1-score against labelled, injected anomaly windows. Built entirely with free and open-source tools, the platform demonstrates that automated, explainable and self-improving incident triage is achievable at zero licensing cost.

Keywords: AIOps, Log Anomaly Detection, Isolation Forest, Retrieval-Augmented Generation, pgvector, Large Language Model, Ollama, Site Reliability Engineering, Docker.

C O N T E N T S

- List of Figures — vi
- List of Tables — vii
- Abbreviations and Symbols — viii

Chapter No.	Topic	Page No.
Chapter 1	Introduction: Background, Problem Statement, Need, Objectives, Scope, Expected Outcomes	1
Chapter 2	Literature Review: Survey table, Thematic Discussion, Research Gap, Novelty/Innovation	5
Chapter 3	Proposed Methodology: Proposed System, Working Principle, Architecture/Block Diagram, Flowchart, Methodology, Components	10
Chapter 4	Project Planning: WBS, Gantt Chart, Roles, Risk Assessment, Milestones	18
Chapter 5	Resources & Feasibility: Technical/Economic/Operational Feasibility, Budget, BOM, Requirements	22
Chapter 6	Expected Results: Prototype behaviour, detection characteristics, performance, test scenarios, dashboard	25
Chapter 7	Sustainability & Impact: Industrial relevance, Societal and Environmental impact, SDG mapping, Future Scope	28
Chapter 8	Conclusion	30
References	IEEE style references	31
Appendices	Architecture diagram, database schema, sample API, dashboard screenshots, docker-compose services, sample .env	33

List of Figures

Figure No.	Figure Name	Page No.
1	Conventional incident triage versus the proposed automated workflow	2
2	System Architecture Diagram	11
3	Flow Chart of the Incident Intelligence Pipeline	11
4	Database Schema (ER) Diagram	15
5	Gantt Chart of the 12-week Project Schedule	19
6	Expected Dashboard Output	27
7	Full-page system architecture diagram	34
8	Database schema diagram (logs, anomalies, incidents)	35
9	Dashboard overview	37
10	Anomalies-by-type chart	37
11	Expanded anomaly row with LLM explanation and matched past incident	38
12	Feedback and system-health view	38

List of Tables

Table No.	Table Name	Page No.
1	Summary of related literature	5
2	Comparison of the proposed platform with typical existing approaches	8
3	Rules used by the rule-based detector	12
4	Factors considered by the severity triage engine	13
5	Database and queue design	14
6	Data flow through the pipeline	15
7	Components and tools used	17
8	Work breakdown structure	18
9	Week-by-week project schedule (Gantt table)	19
10	Roles and responsibilities	20
11	Risk assessment	20
12	Project milestones	21
13	Budget estimation	23
14	Software bill of materials	23
15	Hardware and software requirements	24
16	Expected characteristics of the detection methods	25
17	Planned test scenarios and expected behaviour	26
18	Mapping of the project to Sustainable Development Goals	28
19	REST endpoints exposed by the API service (illustrative)	36
20	Services orchestrated by docker-compose	39

Abbreviations and Symbols

- **AI**: Artificial Intelligence
- **AIOps**: Artificial Intelligence for IT Operations
- **API**: Application Programming Interface
- **BOM**: Bill of Materials
- **CPU**: Central Processing Unit
- **CSV**: Comma-Separated Values
- **DB**: Database
- **DFD**: Data Flow Diagram
- **ER**: Entity–Relationship
- **ETL**: Extract, Transform, Load
- **F1**: F1-score (harmonic mean of precision and recall)
- **GPU**: Graphics Processing Unit
- **HTTP**: Hypertext Transfer Protocol
- **IDE**: Integrated Development Environment
- **JSON / JSONB**: JavaScript Object Notation / binary JSON storage type in PostgreSQL
- **LLM**: Large Language Model
- **LSTM**: Long Short-Term Memory
- **ML**: Machine Learning
- **MTTR**: Mean Time To Resolve (or Recover)
- **NLP**: Natural Language Processing
- **RAG**: Retrieval-Augmented Generation
- **REST**: Representational State Transfer
- **SaaS**: Software as a Service
- **SDG**: Sustainable Development Goal
- **SLA**: Service Level Agreement
- **SQL**: Structured Query Language
- **SRE**: Site Reliability Engineering
- **UI**: User Interface
- **URL**: Uniform Resource Locator
- **WBS**: Work Breakdown Structure
- **YAML**: YAML Ain't Markup Language

- **TP**: True Positives — anomalies correctly detected
- **FP**: False Positives — normal events wrongly flagged
- **FN**: False Negatives — anomalies missed by the detector
- **P**: Precision = TP / (TP + FP)

- **R**: Recall = TP / (TP + FN)
- **F1**: 2 × P × R / (P + R)
- **cos(θ)**: Cosine similarity between two embedding vectors
- **<=>**: pgvector cosine-distance operator (distance = 1 − cosine similarity)
- **d**: Embedding dimension (d = 384 for all-MiniLM-L6-v2)
- **s(x, n)**: Isolation Forest anomaly score of point x for a sample of size n

Chapter 1: Introduction

1.1 Background

Modern software services are built from many cooperating components — web servers, application services, databases, caches and message queues — and each of them continuously writes logs. As a system grows, the volume of these logs grows with it, and an engineer who is paged at night is expected to find the one meaningful signal among millions of lines. Site Reliability Engineering (SRE) is the discipline that treats this operational work as an engineering problem, and **AIOps** (Artificial Intelligence for IT Operations) is the application of machine learning and data analytics to automate parts of it [1], [2].

Two recent developments make a new generation of AIOps tools practical. First, Retrieval-Augmented Generation (RAG) allows a language model to ground its answers in an external, updatable knowledge base rather than relying only on its training data [3]. Second, open-weight large language models (LLMs) such as Llama can now run locally through tools such as Ollama, which removes the need to send sensitive operational data to a third-party cloud service. Together with vector search in PostgreSQL through the pgvector extension, they allow an incident-triage assistant to be built entirely from free and open-source components.

1.1.1 From Monitoring to AIOps

Early operations practice relied on static dashboards and fixed thresholds: an alert fired when CPU usage, error rate or latency crossed a limit chosen by an engineer. As systems moved to microservices and cloud platforms, the number of metrics, logs and traces grew faster than any team could inspect by hand, and thresholds became hard to tune. AIOps emerged to apply anomaly detection, correlation and, more recently, language models to this data so that machines handle the repetitive part of operations and engineers focus on decisions [1].

1.1.2 Role of Logs in Incident Response

Logs are the most detailed record of what a system actually did. Each entry typically carries a timestamp, the route or component involved, a user or request identifier, a status code, a response time and a free-text message. During an incident, engineers search these records to find the first sign of failure, to see which routes and users are affected, and to compare the situation with earlier incidents. Because this reading is manual, its speed and

quality depend on who is on call. The platform proposed in this synopsis targets exactly this step.

1.2 Problem Statement

1.2.1 Existing Problem

Most monitoring tools in use today rely on simple threshold alerts. They tell an engineer **that** something has broken (for example, that latency has crossed a limit), but not **what it means** or **whether a similar incident has occurred before and how it was fixed**. Triage is therefore manual, slow and dependent on tribal knowledge that leaves the organisation when experienced engineers move on. Rule-based detection is precise but misses novel failures, while purely statistical detection finds unusual behaviour but cannot explain it. There is a need for a system that detects anomalies, remembers past incidents, explains new anomalies in plain language and learns from the corrections of its operators.

1.2.2 Industrial Issue

Industrial studies of production incidents report that detection is only one part of the cost: diagnosing the cause and choosing a mitigation take a large share of the time between the first alert and recovery [15], [17]. Teams also face alert fatigue when many low-value alerts compete with the few that matter, and they lose knowledge each time an experienced engineer leaves, because past fixes are scattered across chat threads, tickets and memory.

1.2.3 Research Gap and Need for Improvement

Existing research gives accurate detectors and, more recently, language-model assistants, but few openly reproducible systems join detection, a retrievable memory of past incidents, plain-language explanation, alerting and operator feedback in a single pipeline that a student team or small company can run locally. This project addresses that gap.

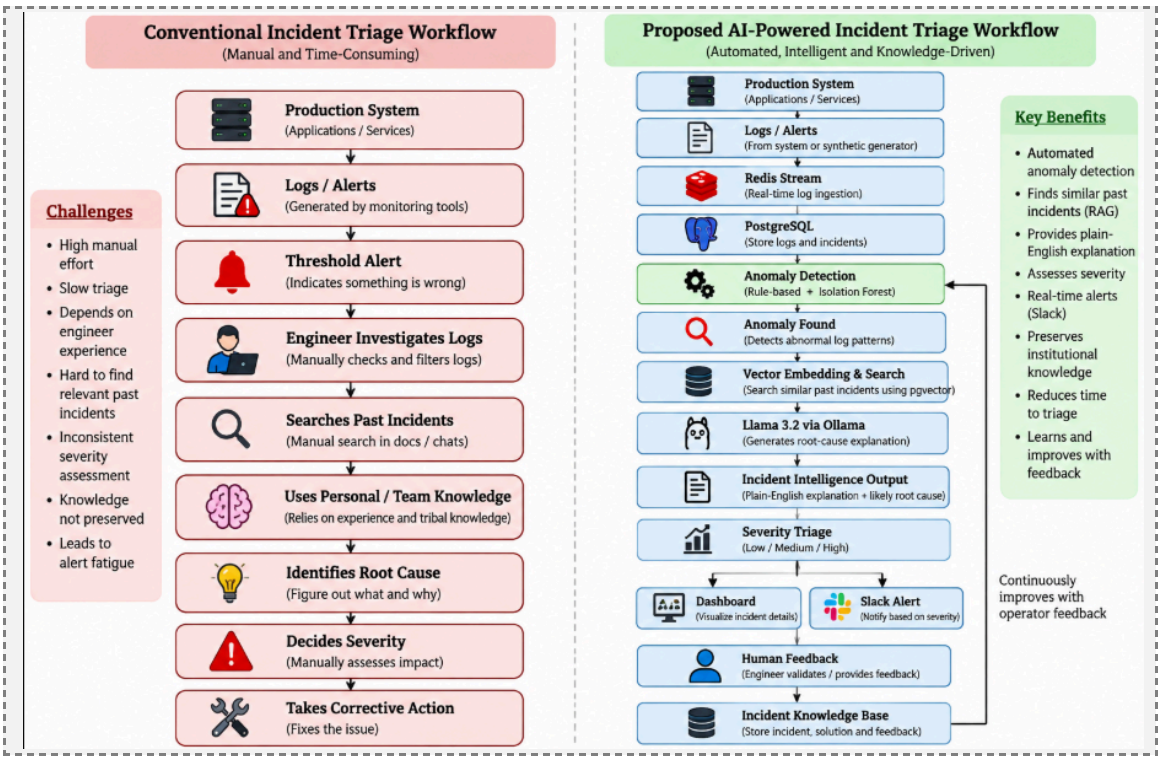


Figure 1: Conventional incident triage versus the proposed automated workflow

1.3 Need of the Project

- **Reduce time-to-triage:** automatically explaining an anomaly shortens the interval between an alert and the first useful diagnosis, which directly influences Mean Time To Resolve (MTTR).
- **Preserve institutional knowledge:** storing past incidents as searchable vector embeddings turns tribal knowledge into a durable, queryable asset.
- **Reduce alert fatigue:** combining explainable rules with a severity triage engine surfaces the important events instead of every raw alert.
- **Keep data private and costs low:** a locally hosted LLM and open-source stack avoid recurring API fees and keep logs inside the organisation's own infrastructure.
- **Improve continuously:** a human feedback loop lets the system learn from real operator corrections rather than remaining static after deployment.

1.4 Objectives

1.4.1 General Objective

To design and implement a fully containerized AIOps platform that automatically detects anomalies in application logs, retrieves similar past incidents, generates a plain-English root-cause explanation, alerts engineers by severity and improves itself from operator feedback.

1.4.2 Specific Objectives

1. Design and implement a real-time log ingestion pipeline capable of accepting both synthetic and external (real) log data.
2. Develop and compare two complementary anomaly detection approaches — explainable rule-based detection and unsupervised machine learning (Isolation Forest).
3. Build a Retrieval-Augmented Generation system using vector embeddings to retrieve semantically similar past incidents.
4. Integrate a locally hosted large language model to automatically generate human-readable incident explanations grounded in retrieved historical context.
5. Implement a feedback-driven mechanism so that the system's knowledge base improves continuously from operator corrections.
6. Containerize the entire multi-service system for reproducible, one-command deployment.

1.5 Scope

1.5.1 In Scope

The project covers the end-to-end **detection → explanation → alerting** pipeline for web-application-style logs: ingestion (synthetic and external), buffering, storage, dual-method anomaly detection, RAG-based incident retrieval, LLM explanation, severity triage, Slack alerting, operator feedback, a REST API and a monitoring dashboard.

1.5.2 Out of Scope

The project does not cover distributed tracing, multi-cloud log aggregation at enterprise scale, or training a custom LLM; it uses an existing open-weight model.

1.5.3 Limitations

The prototype is designed around structured web-application logs and is evaluated mainly on synthetic data with injected anomalies, so its behaviour on the full variety of real production logs must still be established. The quality of explanations depends on the size of the local model and on how many and how well-described the stored past incidents are. The system is deployed on a single node, and no formal security audit or large-scale load test is within the scope of this work.

1.5.4 Future Expansion

Possible extensions include public cloud deployment, multi-tenant support and integration with real company log sources.

1.6 Expected Outcomes

- A fully working, containerized **software prototype** (not a physical prototype) of the incident intelligence pipeline.
- A functioning dashboard showing pipeline health, anomaly statistics, and LLM-generated explanations with matched past incidents.
- A measurable evaluation methodology: a dedicated evaluation script compares detector output against labelled ground-truth anomaly windows and computes precision, recall and F1-score for the rule-based detectors and the ML detector separately.
- The potential for a future startup or SaaS product built around automated incident triage for engineering teams.

Chapter 2: Literature Review

2.1 Introduction

This chapter reviews the research that the proposed platform builds on: log parsing and anomaly detection, unsupervised outlier detection, retrieval-augmented generation and text embeddings, and AIOps for incident management. The aim is to identify what existing work already provides and which gaps remain for an integrated, explainable and self-improving triage system.

Note: all citations in this chapter and in the References section are to be verified by the student against the original publications (authors, year, venue) before final submission.

2.2 Literature Review Table

Table 1: Summary of related literature

Paper	Author	Year	Method	Findings	Research Gap
DeepLog: Anomaly Detection and Diagnosis from System Logs through Deep Learning	Du, Li, Zheng, Srikumar	2017	LSTM predicts the next log key; workflow models for diagnosis	Detects anomalies online and supports diagnosis from execution paths	Needs normal-data training; no natural-language explanation or incident memory
LogAnomaly: Unsupervised Detection of Sequential and Quantitative Anomalies in Unstructured Logs	Meng et al.	2019	template2vec semantic vectors with LSTM	Handles sequential and quantitative anomalies and unseen templates	Heavy training; output is a label, not an explanation
Robust Log-based Anomaly Detection on Unstable Log Data (LogRobust)	Zhang et al.	2019	Semantic log vectors with attention-based Bi-LSTM	Robust to evolving log statements and noise	Requires labelled data; no root-cause narrative or retrieval
Drain: An Online Log Parsing Approach with Fixed Depth Tree	He, Zhu, Zheng, Lyu	2017	Fixed-depth parse tree for online template mining	Accurate and efficient streaming log parsing	Parsing only; no detection or explanation stage

Paper	Author	Year	Method	Findings	Research Gap
Experience Report: System Log Analysis for Anomaly Detection	He, Zhu, He, Lyu	2016	Comparative study of supervised and unsupervised log methods	Shows trade-offs between supervised and unsupervised approaches	Offline evaluation; not an operational pipeline
Loghub: A Large Collection of System Log Datasets towards Automated Log Analytics	Zhu et al.	2020	Public benchmark collection of system logs	Standard datasets for reproducible log research	Datasets only; no incident knowledge base
Log-based Anomaly Detection with Deep Learning: How Far Are We?	Le, Zhang	2022	Re-evaluation of deep-learning detectors under varied settings	Reported performance is sensitive to data and settings	Highlights need for robust, explainable detection
Isolation Forest	Liu, Ting, Zhou	2008	Random trees isolate outliers via short path lengths	Linear time, low memory, no labels required	Flags outliers but gives no semantic reason
Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks	Lewis et al.	2020	Dense retriever combined with a seq2seq generator	Updatable non-parametric memory improves factual generation	Evaluated on general NLP tasks, not operations incidents
Dense Passage Retrieval for Open-Domain Question Answering	Karpukhin et al.	2020	Dual-encoder dense vector retrieval	Outperforms sparse BM25 retrieval on QA	Text QA focus; not applied to log or incident data
Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks	Reimers, Gurevych	2019	Siamese BERT yielding sentence embeddings for cosine similarity	Fast, meaningful semantic similarity search	General text; not specialised to logs
MiniLM: Deep Self-Attention Distillation for Task-Agnostic Compression	Wang et al.	2020	Distillation into small Transformer models	Compact model with strong accuracy; basis of all-MiniLM models	General-purpose, not domain tuned
A Survey of AIOps Methods for Failure Management	Notaro, Cardoso, Gerndt	2021	Systematic literature survey	Taxonomy of failure prediction, detection and root-cause analysis	Few end-to-end, explainable systems with incident memory

Paper	Author	Year	Method	Findings	Research Gap
Recommending Root-Cause and Mitigation Steps for Cloud Incidents using Large Language Models	Ahmed et al.	2023	LLMs applied to historical incident data	LLMs can suggest useful root causes and mitigations	Proprietary cloud-scale data and models; no operator feedback loop
RAGLog: Log Anomaly Detection using Retrieval Augmented Generation	Pan, Wong, Yuan	2023	RAG with a vector store and LLM for log anomaly detection	Shows feasibility of RAG for log analysis	Detection-centred; no hybrid detectors, alerting or feedback learning
How to Fight Production Incidents? An Empirical Study on a Large-scale Cloud Service	Ghosh et al.	2022	Empirical study of production incidents	Triage and mitigation are major time sinks	Study only; does not propose automated memory-based triage

2.2.1 Thematic Discussion

Log parsing. Raw log lines must be converted into structured templates before they can be analysed. Drain parses logs online with a fixed-depth tree and became a standard baseline, and the Loghub collection gave researchers shared datasets for comparison [7], [14]. The proposed platform uses the Drain3 implementation to mine templates from incoming messages and therefore builds on this line of work.

Log anomaly detection. DeepLog models normal execution as a sequence and flags deviations, LogAnomaly adds semantic information about templates, and LogRobust copes with changing log statements [4]–[6]. A comparative study and a later re-evaluation both show that results depend strongly on data and settings, and that simple methods remain competitive [8], [13]. This supports the decision to combine transparent rules with an unsupervised Isolation Forest rather than to rely on a single deep model.

Embeddings and retrieval. Sentence-BERT and MiniLM produce compact sentence vectors that can be compared by cosine similarity, and dense retrieval outperforms keyword matching on many question-answering tasks [10]–[12]. RAG combines such retrieval with a generator so that answers are grounded in stored documents [3]. In this project the stored documents are past incidents.

AIOps and language models. The AIOps survey organises failure management into prediction, detection and root-cause analysis [1]. Recent work applies large language models to recommend root causes and mitigations from historical incidents, and RAGLog explores retrieval-augmented log analysis [15], [16]. These studies are encouraging but rely on proprietary data or models, or concentrate on detection alone.

Table 2: Comparison of the proposed platform with typical existing approaches

Capability	Threshold monitoring	Deep-learning log detectors	Proposed platform
Detects anomalies	Yes, for pre-set thresholds only	Yes, learned from log sequences	Yes, using rules and Isolation Forest
Explains the anomaly in plain language	No	Usually no (label or score only)	Yes, using a local LLM
Recalls similar past incidents	No	No	Yes, using pgvector retrieval (RAG)
Learns from operator feedback	No	Rarely	Yes, corrections are written to the incident store
Alerting by severity	Basic	Usually outside scope	Yes, with Slack alerts
Runs locally at no licence cost	Depends on product	Research code	Yes, open-source stack in Docker Compose

2.3 Research Gap

The literature shows strong progress in each individual building block. Deep-learning detectors such as DeepLog, LogAnomaly and LogRobust achieve high detection quality but return only an anomaly label [4]–[6]. Classical unsupervised methods such as Isolation Forest are cheap and need no labels, but their output has no semantic meaning for an operator [9]. RAG and sentence embeddings provide a way to ground language-model output in a growing knowledge base [3], [10]–[12], and early work such as RAGLog applies it to logs [16]. Industrial studies show that triage and mitigation dominate incident handling time [15], [17].

The following gaps remain:

- Most detectors **detect without explaining**: the engineer still has to work out what the anomaly means.
- Few systems combine **explainable rules and unsupervised ML** and compare them quantitatively.

- Past-incident knowledge is rarely used as a **retrievable memory** during live triage, and almost never **updated from operator feedback**.
- Industrial LLM-based approaches depend on proprietary data and hosted models, which limits reuse by students, start-ups and privacy-sensitive teams.

2.4 Novelty / Innovation

- An **integrated pipeline** from ingestion to explanation to alerting, containerized as one reproducible system.
- **Dual detection** — deterministic rules plus Isolation Forest — with a labelled evaluation of the precision/recall trade-off between them.
- **Incident memory through RAG** using pgvector inside the same PostgreSQL database that stores the logs.
- **Local LLM explanations** (Ollama, Llama 3.2) grounded in the retrieved past incident, with no cloud dependency.
- **Self-improving knowledge base**: operator corrections are embedded and written back to the incident store.
- **Zero licensing cost**: the entire stack is free or open source.

Chapter 3: Proposed Methodology

3.1 Proposed System

The proposed system is a fully containerized, multi-service pipeline that converts raw logs into explained, prioritised incidents. It consists of the following stages: (i) ingestion from a synthetic log generator or an external source; (ii) buffering in a Redis Stream and persistence in PostgreSQL; (iii) two independent anomaly detectors, one rule-based and one machine-learning based; (iv) RAG-based retrieval of similar past incidents from a pgvector store; (v) LLM-based explanation generation; (vi) severity triage and Slack alerting; (vii) a human feedback loop that grows the incident memory; and (viii) a FastAPI backend with a React dashboard.

3.2 Working Principle

1. Logs arrive from the synthetic generator (which injects anomalies in labelled ground-truth windows) or from an external system through the API-key-secured **/ingest** endpoint, and are pushed into the Redis Stream **logs_stream**.
2. The consumer service reads the stream and writes structured records into the PostgreSQL **logs** table.
3. The rule-based detector and the Isolation Forest detector independently scan new logs and write anomalies, tagged by detector, into the shared **anomalies** table.
4. For each anomaly, the message is embedded with all-MiniLM-L6-v2 and compared by cosine similarity (pgvector **<=>** operator) with the embeddings in the **incidents** table; the closest past incident is retrieved.
5. The anomaly and the retrieved incident are given to the local Llama 3.2 model through Ollama, which writes a short plain-English explanation.
6. The triage engine assigns a severity (critical, warning or info); high-severity alerts are posted to Slack through an Incoming Webhook.
7. Engineers review anomalies on the dashboard and mark explanations as correct or incorrect; the feedback processor embeds the corrections and writes them to the **incidents** table, improving future retrieval.

3.3 System Architecture / Block Diagram

The architecture is organised as loosely coupled services that communicate through Redis and PostgreSQL. Producers and the database writer are decoupled by the stream, the two detectors run in parallel and independently, and the explanation, alerting and feedback services each operate on the shared anomalies and incidents tables. The API and dashboard sit on top of the database as the operator-facing layer.

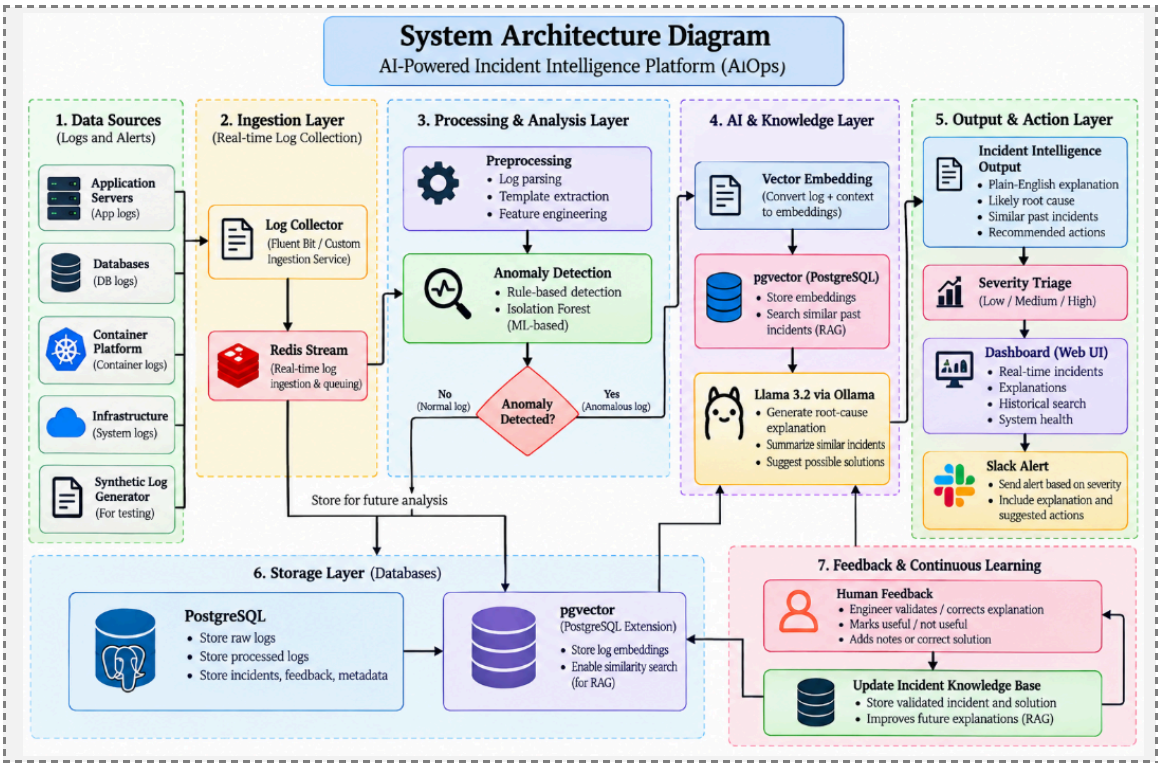


Figure 2: System Architecture Diagram

3.4 Flow Chart

The flow chart shows the path of a single log record from ingestion to alert and feedback, including the decision points for anomaly detection, incident match and severity.

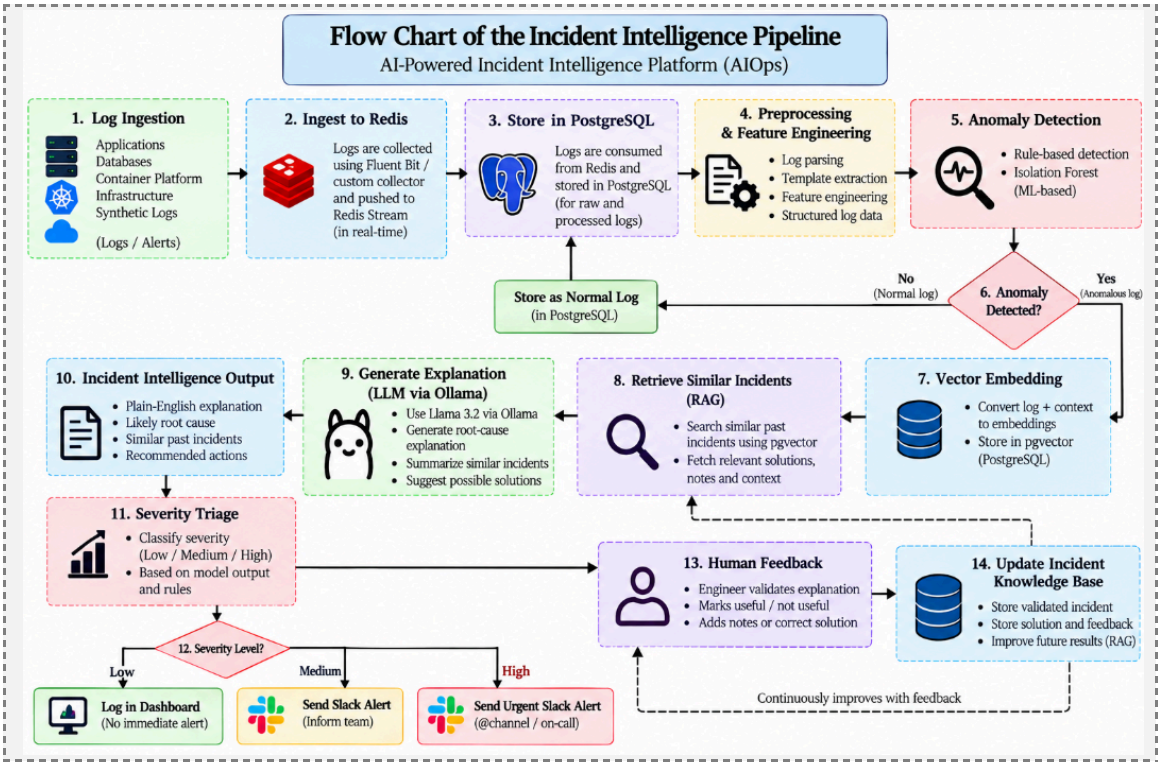


Figure 3: Flow Chart of the Incident Intelligence Pipeline

3.5 Methodology

3.5.1 Log Ingestion and Storage

A synthetic generator simulates web-application traffic (routes, users, status codes and response times) and injects anomalies inside labelled windows so that detector output can later be scored against ground truth. External systems can send real logs to the **/ingest** REST endpoint, which is protected by an API key supplied in a request header. All accepted records are appended to the Redis Stream **logs_stream**. A consumer service reads the stream and persists structured records into PostgreSQL. This design decouples producers from the database writer, so that bursts of traffic or a slow database do not cause log loss at the source.

3.5.2 Dual-Method Anomaly Detection

Two detectors run in parallel as independent services and write to the same anomalies table, with the **rule_matched** field recording which detector flagged the event.

Rule-based detector. Deterministic, explainable rules identify known failure patterns, supported by Drain3 log template mining that groups messages into templates (see the table below).

Table 3: Rules used by the rule-based detector

Rule	Condition	Meaning
error_status_code	HTTP 5xx status code	Server-side failures
latency_spike	response_time_ms above a configured threshold	Performance degradation
crash_pattern	Crash or resource-exhaustion keywords in the message text	Crashes, out-of-memory and similar failures

Machine-learning detector. An unsupervised Isolation Forest (scikit-learn) is trained continuously on a rolling window of recent logs using features such as response time, status code and a missing-latency indicator. It isolates statistical outliers — events that no rule anticipates — by their short path length in random trees [9]. Because it needs no labels, it adapts as normal behaviour drifts.

The Isolation Forest builds many random trees on subsamples of the data. Points that can be separated from the rest after only a few random splits receive a short average path length and a high anomaly score, defined as s(x, n) = 2^(−E[h(x)] / c(n)), where E[h(x)] is the average path length of x over the trees and c(n) is the average path length of an unsuccessful search in a binary search tree of n points [9]. Scores close to 1 indicate likely anomalies. Because the model is retrained on a rolling window, the notion of normal follows the recent behaviour of the system.

3.5.3 Retrieval-Augmented Generation for Incident Memory

A library of past incidents is stored in the PostgreSQL **incidents** table using the pgvector extension. Each incident description is embedded into a 384-dimension vector with the sentence-transformer model all-MiniLM-L6-v2. When a new anomaly occurs, its message is embedded in the same way and compared by cosine similarity (the **<=>** operator) with the stored vectors to retrieve the most relevant past incident. The system can therefore state that an event resembles an incident seen before, rather than merely that it is unusual [3], [10].

For two embedding vectors a and b of dimension d = 384, cosine similarity is cos(θ) = (a · b) / (‖a‖ ‖b‖). The pgvector <=> operator returns the cosine distance, 1 − cos(θ), so the stored incident with the smallest distance is the best match. Performing this search inside PostgreSQL keeps logs, anomalies and incident memory in one database and avoids a separate vector service [21].

3.5.4 LLM-Based Explanation Generation

The retrieved incident and the new anomaly are placed in a prompt for Llama 3.2 running locally under Ollama. The model produces a short, plain-English explanation of what likely went wrong and how it relates to the matched incident. Grounding the prompt in a retrieved record, instead of asking the model to guess, is intended to keep explanations relevant and reduce unsupported statements. Running the model locally keeps operational data inside the deployment.

3.5.5 Severity Triage and Alerting

A lightweight, explainable rules engine classifies each anomaly as **critical**, **warning** or **info** from its status code, latency and message content. High-severity alerts are pushed in real time to a Slack channel through Incoming Webhooks.

The factors considered by the severity triage engine are summarised below.

Table 4: Factors considered by the severity triage engine

Factor	How it influences severity
Status code	Server-side errors (5xx) indicate a failure on the service side and raise severity
Latency	Response times far above the normal range indicate degradation and raise severity
Message content	Crash or resource-exhaustion keywords indicate serious faults and raise severity
Detector source	The rule_matched field records whether a rule or the ML detector flagged the event, helping operators judge confidence

3.5.6 Human Feedback Loop

Engineers can mark an explanation as correct or incorrect from the dashboard, optionally with a correction note. The feedback processor embeds the corrections and writes them back into the incidents vector store, so the retrieval memory improves from real operator knowledge and closes the loop between human expertise and the AI knowledge base.

3.5.7 Backend API and Dashboard

A FastAPI service exposes REST endpoints for statistics, recent anomalies, recent logs, system health, feedback submission and external log ingestion. A React (Vite) single-page dashboard polls the API and displays live health indicators per pipeline stage,

summary cards, a chart of anomalies by type (Recharts), and an expandable, searchable anomaly table whose rows reveal the LLM explanation and the matched past incident.

3.5.8 Containerization

The entire system of more than ten services — Redis, PostgreSQL with pgvector, consumer, generator, rule-based analyzer, ML detector, RAG/LLM explainer, triage and alerting, feedback processor, API and dashboard — is orchestrated by a single **docker-compose.yml** with health checks and automatic restarts, so the platform starts with one command instead of ten or more manually managed terminals.

3.5.9 Database Design

Table 5: Database and queue design

Store	Main fields	Purpose
logs	id, timestamp, route, user_id, status_code, response_time_ms, message, raw_json (JSONB)	Raw and parsed application log records
anomalies	id, log reference, rule_matched (detector source), explanation text, matched_incident reference, severity, feedback_correct, correction_note, feedback_processed	Anomalies from both detectors with explanation, severity and operator feedback
incidents	incident description, 384-dimension pgvector embedding column	RAG knowledge base of past incidents; grows through the feedback loop
logs_stream (Redis Stream)	Stream entries carrying log payloads	Transient queue decoupling producers from the database writer

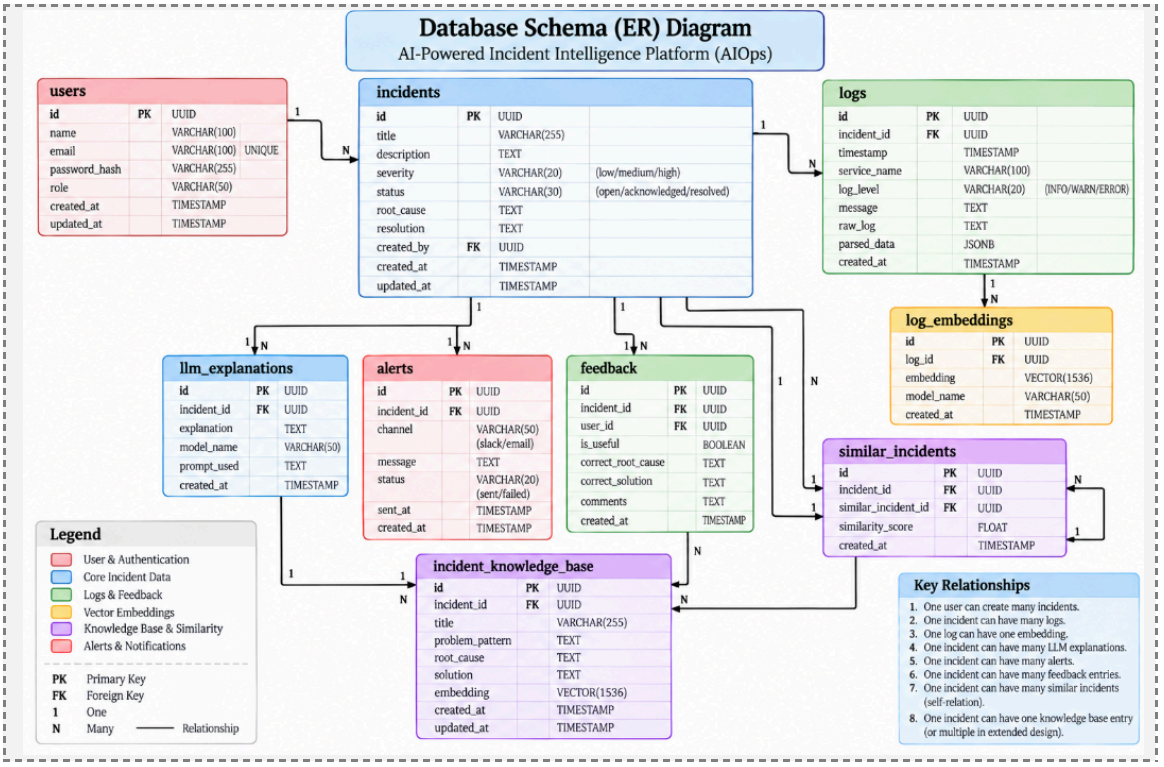


Figure 4: Database Schema (ER) Diagram

Table 6: Data flow through the pipeline

Stage	Input	Processing	Output
Ingestion	Synthetic or external log record	Authentication (external only); push to stream	Entry in logs_stream
Persistence	Stream entry	Parse into structured fields	Row in logs table
Detection	New log rows	Rules and Drain3; Isolation Forest scoring	Row in anomalies table
Retrieval	Anomaly message	Embed; cosine search in incidents	Matched past incident
Explanation	Anomaly and matched incident	Local LLM generation	Explanation text
Triage and alert	Explained anomaly	Severity rules; Slack webhook	Severity; alert message
Feedback	Operator correction	Embed and store	New row in incidents

3.5.10 Evaluation Methodology

A dedicated evaluation script compares each detector's output with the labelled, injected anomaly windows and computes **precision** (P = TP / (TP + FP)), **recall** (R = TP / (TP + FN)) and **F1-score** (F1 = 2PR / (P + R)) separately for the rule-based detectors and for the ML detector. The evaluation is designed to expose the classic trade-off between

explainable rule-based detection (expected high precision, narrower recall) and unsupervised ML detection (expected higher recall, more false positives), giving a data-driven rather than anecdotal comparison.

3.5.11 Algorithm Summary

The overall control flow of the pipeline can be summarised in the following pseudocode.

```
for each log record received (generator or /ingest):
    push record to logs_stream
consumer: read stream -> insert row into logs
for each new log row (rule detector and ML detector in parallel):
    if rule matches OR isolation_forest_score(row) is high:
        insert anomaly (rule_matched = detector name)
for each anomaly without explanation:
    v = embed(anomaly.message)
    incident = nearest incident to v by cosine distance
    anomaly.explanation = LLM(anomaly, incident)
    anomaly.severity = triage(anomaly)
    if anomaly.severity is high: send Slack alert
for each anomaly with operator feedback not yet processed:
    insert embedded correction into incidents; mark processed
```

3.5.12 Security and Privacy Considerations

The external ingestion endpoint accepts requests only with a valid API key in a request header. Secrets such as the API key, database password and Slack webhook URL are supplied through environment variables and are never stored in source code. Because the LLM and embedding model run locally, log content is not sent to any third-party AI service. Alerts posted to Slack should avoid carrying sensitive fields; this should be reviewed for each deployment.

3.6 Components / Materials

The project is purely software. The table below lists every tool, library and service used.

Table 7: Components and tools used

Component	Specification	Quantity / Version
Programming language	Python	3.11
Web framework / server	FastAPI, Uvicorn	[version]
Relational database	PostgreSQL with pgvector extension	16
Message queue	Redis (Streams)	[version]
Anomaly detection (ML)	scikit-learn — Isolation Forest	[version]

Component	Specification	Quantity / Version
Embeddings	sentence-transformers — all-MiniLM-L6-v2 (384-dim)	[version]
Log template mining	Drain3	[version]
LLM runtime and model	Ollama; Llama 3.2	[version]
Alerting	Slack Incoming Webhooks	1 channel
Frontend	React (Vite), Recharts	[version]
Infrastructure	Docker, Docker Compose	[version]
Development tooling	Cursor / Antigravity IDE; Git and GitHub	1 each

Chapter 4: Project Planning

4.1 Work Breakdown Structure

The project was planned as a solo twelve-week build divided into phases. Each phase produced a working, testable increment of the pipeline.

Table 8: Work breakdown structure

No.	Phase	Activities	Duration	Deliverable
1	Requirements and System Design	Problem study, architecture, database design, tool selection	Week 1	Architecture and schema
2	Ingestion and Storage	Synthetic generator, /ingest endpoint, Redis Stream, consumer, PostgreSQL logs table	Weeks 2–3	Working log pipeline
3	Anomaly Detection	Rule-based detector with Drain3; Isolation Forest detector; anomalies table	Weeks 3–5	Two detectors
4	RAG Incident Memory	pgvector setup, embeddings with all-MiniLM-L6-v2, similarity retrieval	Weeks 5–6	Incident retrieval
5	LLM Explanation	Ollama with Llama 3.2, prompt design, explanation service	Weeks 6–7	Explanations per anomaly
6	Triage and Alerting	Severity rules engine, Slack webhook integration	Weeks 7–8	Real-time alerts
7	Feedback Loop	Feedback fields and processor, write-back of embeddings	Weeks 8–9	Self-improving memory
8	API and Dashboard	FastAPI endpoints, React dashboard, charts, anomaly table	Weeks 7–10	Operator interface
9	Containerization	Dockerfiles, docker-compose, health checks, restart policy	Weeks 10–11	One-command deployment
10	Evaluation	Labelled windows, evaluation script for precision, recall and F1	Weeks 11–12	Detector comparison
11	Documentation	Synopsis, report, diagrams, README	Weeks 10–12	Project documents

4.2 Gantt Chart

The table below shows the week-by-week schedule; shaded cells mark the weeks in which each phase was active. A visual Gantt chart may be inserted in the placeholder below.

Table 9: Week-by-week project schedule (Gantt table)

Phase	W1	W2	W3	W4	W5	W6	W7	W8	W9	W10	W11	W12
Design												

Phase	W1	W2	W3	W4	W5	W6	W7	W8	W9	W10	W11	W12
Ingestion & storage												
Anomaly detection												
RAG memory												
LLM explanation												
Triage & alerting												
Feedback loop												
API & dashboard												
Containerization												
Evaluation												
Documentation												

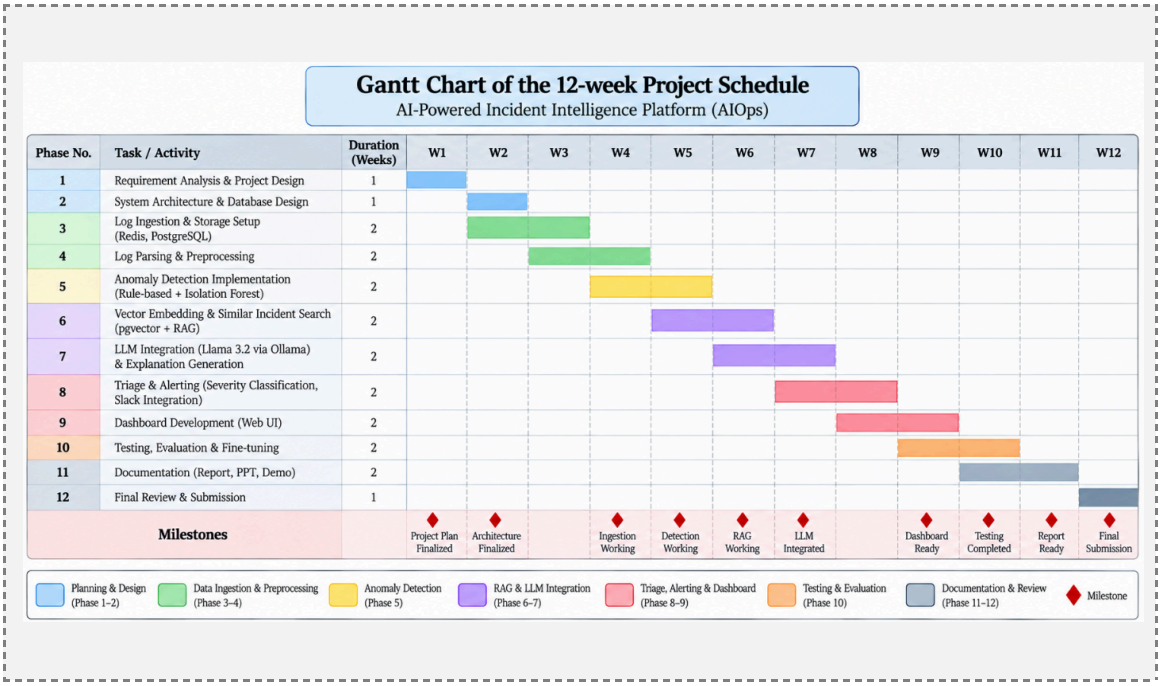


Figure 5: Gantt Chart of the 12-week Project Schedule

4.3 Roles and Responsibilities

This was a solo project, so all roles were performed by the sole developer. The functional roles are listed in the format expected by the template.

Table 10: Roles and responsibilities

Functional Role	Responsibilities	Assigned To
System Design	Architecture, service boundaries, database schema, technology selection	Sole developer

Functional Role	Responsibilities	Assigned To
Backend Development	Ingestion, consumer, detectors, triage, alerting, FastAPI API, React dashboard	Sole developer
ML / Data Science	Feature design, Isolation Forest, embeddings, RAG retrieval, prompt design, evaluation	Sole developer
DevOps / Containerization	Dockerfiles, docker-compose, health checks, version control	Sole developer
Documentation	Synopsis, diagrams, README, report	Sole developer

4.4 Risk Assessment

Table 11: Risk assessment

Risk	Impact	Mitigation
Data pipeline backlog under bursty traffic	High	Buffer logs in a Redis Stream so producers are decoupled from the database writer; monitor stream length on the health view.
High false-positive rate from the ML detector	Medium	Use rule-based detection alongside it, tag each anomaly by detector, tune contamination and features, and evaluate with labelled windows.
Container orchestration failures	High	Health checks, automatic restarts and ordered start-up in docker-compose.
Local LLM latency or poor resource availability	Medium	Use a compact model (Llama 3.2), keep prompts short, process explanations asynchronously from detection.
Irrelevant or unsupported LLM explanations	Medium	Ground prompts in the retrieved incident and let operators correct explanations through feedback.
Poor retrieval quality with a small incident library	Medium	Seed an initial incident library and grow it via the feedback loop.
Alert noise or Slack delivery failure	Low	Alert only on high severity; log delivery errors and keep alerts visible on the dashboard.
Unauthorised use of the /ingest endpoint	Medium	Require an API key header and keep secrets in environment variables, not in source code.
Time overrun on a solo 12-week schedule	Medium	Phase-wise milestones with core pipeline first and optional features last.

4.5 Milestones and Deliverables

Table 12: Project milestones

Milestone	Target week	Deliverable
M1: Design complete	Week 1	Architecture, database schema and tool selection
M2: Data path working	Week 3	Logs flow from generator and /ingest into PostgreSQL

Milestone	Target week	Deliverable
M3: Detection working	Week 5	Rule-based and Isolation Forest detectors writing anomalies
M4: Explanation working	Week 7	Retrieval of past incidents and LLM explanations
M5: Alerts and feedback	Week 9	Severity triage, Slack alerts and feedback write-back
M6: Full system	Week 11	Dashboard, API and one-command Docker Compose deployment
M7: Evaluation and documents	Week 12	Precision, recall and F1 comparison; documentation

4.6 Project Management Approach

The build followed an incremental approach: each phase ended with a working service that could be run and tested on its own before the next was started. Source code was kept under Git with GitHub as the remote repository, which gave a history of changes and a safe way to undo mistakes. Services were kept small and communicate only through Redis and PostgreSQL, so that one component could be replaced or restarted without affecting the others.

Chapter 5: Resources and Feasibility

5.1 Technical Feasibility

Every component of the platform is mature and widely used: FastAPI and PostgreSQL for the service and data layers, Redis Streams for buffering, scikit-learn for Isolation Forest, sentence-transformers for embeddings, pgvector for similarity search, and Docker Compose for orchestration. Ollama makes it practical to run a compact open-weight model such as Llama 3.2 on a developer-class machine. The integration points between components are standard (HTTP, SQL and Redis protocols), which makes the system technically feasible for one developer within twelve weeks.

5.2 Economic Feasibility

The project was built entirely on free-tier and open-source tools, so there is no licensing cost. Local LLM inference removes per-request API fees. The only potential future cost is optional cloud hosting if the platform is later deployed outside the development machine.

5.3 Operational Feasibility

The whole system starts with a single docker-compose command, with health checks and automatic restarts, which keeps day-to-day operation simple. Operators interact through a browser dashboard and Slack alerts, tools that engineering teams already use. The feedback loop requires only a click to mark an explanation correct or incorrect, so adoption effort for operators is low.

5.4 Budget Estimation

Table 13: Budget estimation

Item	Description	Cost (USD)
Programming language and runtime	Python 3.11	$0
Backend, data and ML libraries	FastAPI, Uvicorn, scikit-learn, sentence-transformers, Drain3	$0
Database and queue	PostgreSQL 16, pgvector, Redis	$0
LLM runtime and model	Ollama with Llama 3.2 (local inference, no API fees)	$0
Frontend	React (Vite), Recharts	$0
Containerization	Docker and Docker Compose	$0

Item	Description	Cost (USD)
Alerting	Slack workspace with Incoming Webhooks (free tier)	$0
Development tools	Cursor / Antigravity IDE (free tier), Git, GitHub	$0
Development machine	Existing personal computer (no additional purchase)	$0
Total licensing and software cost		**$0**
Future / optional: cloud hosting	Hosting on a public cloud VM for demonstration or pilot use [estimate when chosen; not incurred]	[Optional, not incurred]

5.5 Bill of Materials

As the project is software-only, the bill of materials lists software and libraries rather than physical parts. Entries marked [version] or [verify] are to be completed by the student from the project's requirements and Docker files.

Table 14: Software bill of materials

Software	Version	Purpose	Licence	Cost
Python	3.11	Implementation language	PSF License	$0
FastAPI / Uvicorn	[version]	REST API and ASGI server	MIT / BSD	$0
PostgreSQL	16	Relational store for logs, anomalies, incidents	PostgreSQL License	$0
pgvector	[version]	Vector similarity search in PostgreSQL	PostgreSQL License	$0
Redis (Streams)	[version]	Log buffering queue	Open-source / source-available (depends on version) [verify]	$0
scikit-learn	[version]	Isolation Forest detector	BSD-3-Clause	$0
sentence-transformers (all-MiniLM-L6-v2)	[version]	384-dim text embeddings	Apache-2.0	$0
Drain3	[version]	Log template mining	MIT [verify]	$0
Ollama	[version]	Local LLM serving	MIT [verify]	$0
Llama 3.2	[model size]	Explanation generation	Llama community licence [verify terms]	$0
React (Vite), Recharts	[version]	Dashboard and charts	MIT	$0
Docker, Docker Compose	[version]	Containerization and orchestration	Apache-2.0 (Engine) [verify]	$0

Software	Version	Purpose	Licence	Cost
Git and GitHub	[version]	Version control	GPL-2.0 (Git); free GitHub plan	$0
Slack Incoming Webhooks	Free tier	Real-time alert delivery	Slack terms of service	$0

5.6 Hardware and Software Requirements

Although the project is software-only, the system needs a machine able to run several containers and a local language model. The figures below are to be completed by the student from the machine actually used.

Table 15: Hardware and software requirements

Item	Requirement	Remark
Processor	Multi-core CPU [specify]	Runs all services and local LLM inference; a GPU is optional and speeds up LLM and embeddings
Memory	[specify] GB RAM	Docker services, PostgreSQL, Redis and the Llama 3.2 model must fit together
Storage	[specify] GB free	Container images, model files and the growing logs table
Operating system	Windows, macOS or Linux with Docker	Docker Compose provides the same environment on each
Software	Docker, Docker Compose, Ollama, Git	Other dependencies are installed inside the containers
Network	Internet for first-time image and model download	Normal operation can run locally; Slack alerts need outbound HTTPS

Chapter 6: Expected Results

In line with the synopsis guidelines, this chapter describes **expected** outcomes only; no measured results are reported here.

6.1 Expected Prototype Behaviour

The prototype is designed to ingest logs continuously from the synthetic generator or the external endpoint, store them without loss, flag anomalies through both detectors, attach a plain-English explanation and the most similar past incident to each anomaly, assign a severity, and send high-severity alerts to Slack. The dashboard is expected to present live pipeline health, summary statistics, a chart of anomalies by type, and an expandable, searchable anomaly table.

6.2 Expected Detection Characteristics

Based on the nature of the two methods, the evaluation against labelled anomaly windows is expected to show the following relationship.

Table 16: Expected characteristics of the detection methods

Aspect	Rule-based detectors	Isolation Forest	Combined platform
Detection precision	Expected to be high: rules fire only on explicit failure patterns	Expected to be lower: statistical outliers include benign rare events	Combined view lets operators weigh each anomaly by its source
Detection recall	Expected to be narrower: only anticipated failure patterns are caught	Expected to be higher: finds unusual behaviour no rule anticipates	Expected to exceed either detector alone
Explainability	Each alert maps to a named rule	Score only; interpretation comes from the LLM explanation	Every anomaly accompanied by a plain-English explanation
False positives	Expected to be few	Expected to be more frequent	Reduced operator impact through severity triage
Adaptation	Static until rules change	Adapts through rolling-window retraining	Memory grows through operator feedback

6.3 Expected System Performance

- The Redis Stream is expected to absorb bursts of logs so that the database writer and detectors can work at their own pace without data loss.
- Explanation generation by the local LLM is expected to add latency per anomaly; it is designed to run asynchronously so that detection and alerting are not blocked.

- Retrieval quality is expected to improve over time as operator corrections are embedded and added to the incident store.
- The complete system is expected to start reproducibly from one command on any machine with Docker installed.

6.4 Expected Test Scenarios

The following scenarios are planned to exercise the pipeline, using injected anomalies with known labels so that behaviour can be checked against ground truth.

Table 17: Planned test scenarios and expected behaviour

Scenario	Condition	Expected behaviour
Normal traffic	Typical routes with 2xx status codes and usual latency	Few or no anomalies; dashboard shows healthy stages
Server error burst	Window of 5xx responses injected	Rule-based detector flags error_status_code; severity raised; Slack alert sent
Latency spike	Response times raised well above normal for a window	latency_spike rule and Isolation Forest flag the window
Crash pattern	Messages with crash or exhaustion keywords	crash_pattern rule flags events; explanation refers to the matched past incident
Unusual pattern without rule	Missing latency or odd combination of fields	Isolation Forest flags events that no rule covers
Operator correction	Engineer marks an explanation incorrect with a note	Correction is embedded and stored; later similar anomalies can retrieve it
External log source	Request to /ingest with valid and invalid API key	Valid key accepted into the stream; invalid key rejected

6.5 Expected Dashboard Output

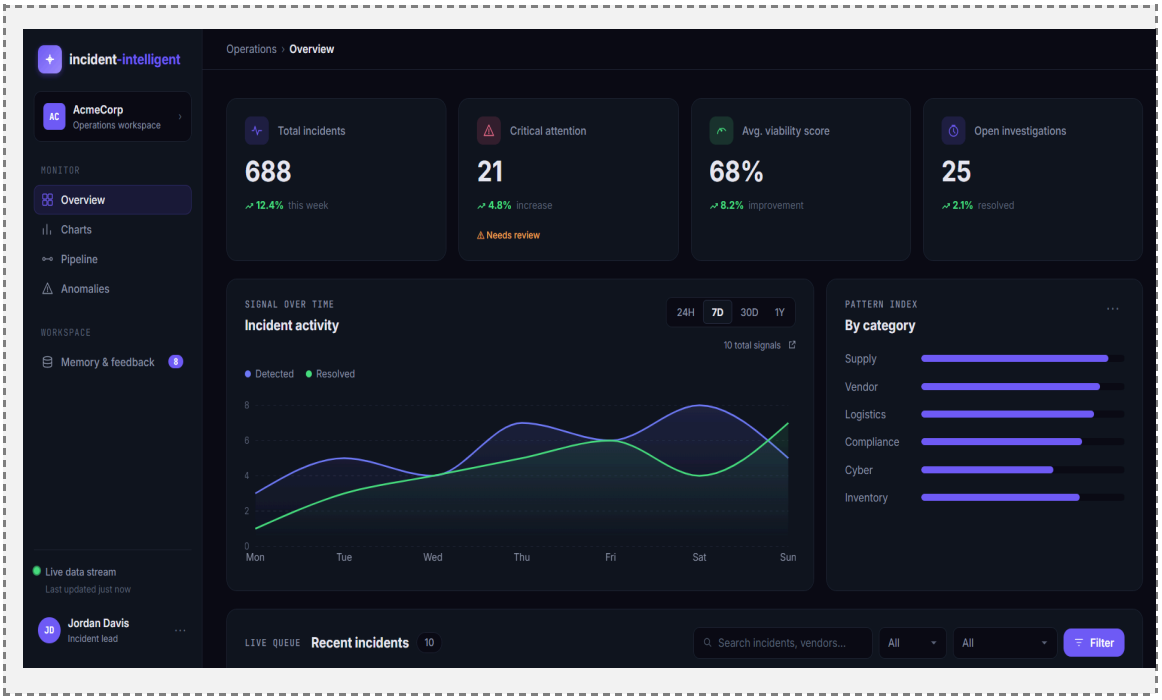


Figure 6: Expected Dashboard Output

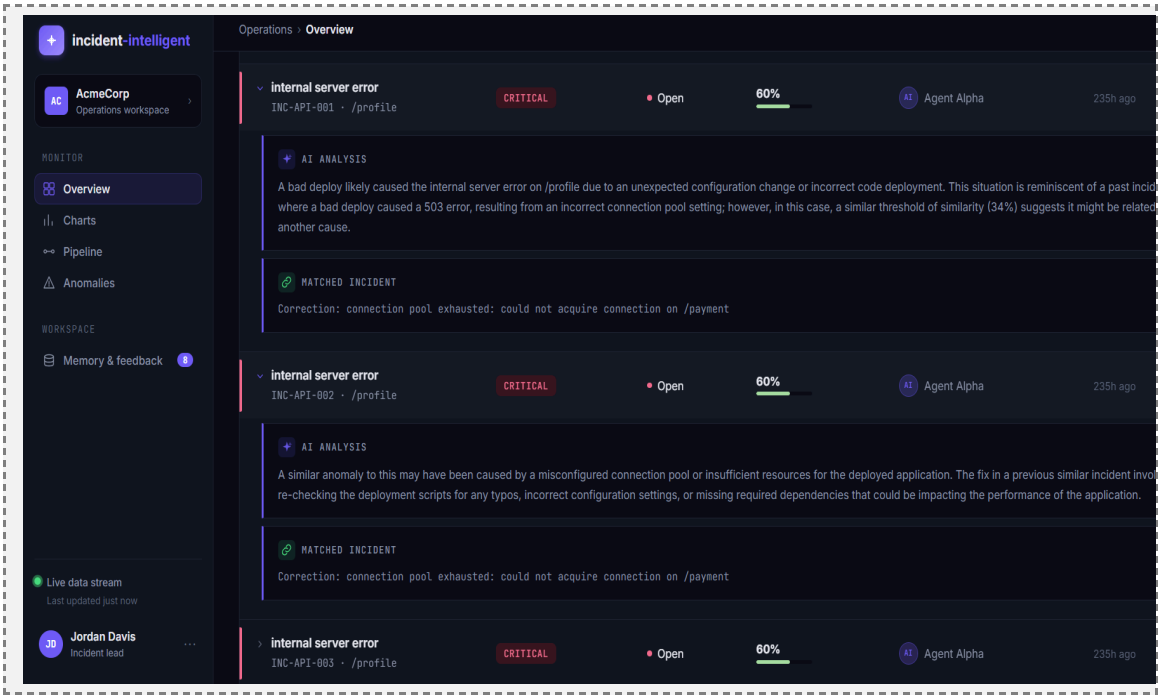


Figure 6: Anomaly Detection and Resolving

Chapter 7: Sustainability and Impact

7.1 Industrial Relevance

Incident management is a core cost centre for any organisation that runs online services. Automating the first step of triage — detection, similar-incident recall and explanation — addresses a problem faced by SRE and DevOps teams across industries, from e-commerce and fintech to SaaS. The same architecture could later be offered as a product for engineering teams that want automated incident triage.

7.2 Societal Impact

Faster diagnosis reduces service downtime for end users and lowers the burden of night-time on-call work for engineers. Capturing past incident knowledge in a searchable store also reduces dependence on a few experienced individuals and helps new engineers learn from earlier incidents. Because the stack is open source and local, small teams, start-ups and educational institutions can adopt it at no cost.

7.3 Environmental Impact

Undetected or slowly diagnosed incidents can waste compute through runaway processes and repeated failures, so quicker triage can reduce this waste. The platform itself consumes energy, mainly for the local LLM and embeddings; this is limited by choosing a compact model, running inference locally on demand and using lightweight rule-based triage instead of repeated large-model calls.

7.4 SDG Mapping

The project contributes most directly to the following United Nations Sustainable Development Goals [29]. Other goals have not been claimed because the link would be indirect.

Table 18: Mapping of the project to Sustainable Development Goals

Goal	Justification
SDG 9 — Industry, Innovation and Infrastructure	Strengthens the reliability of digital infrastructure and promotes innovation through an open-source, reusable AIOps platform that smaller teams can adopt without licence costs.
SDG 8 — Decent Work and Economic Growth	Reduces repetitive, stressful on-call triage work and improves productivity; helps organisations avoid revenue and service losses from slow incident handling.

Goal	Justification
SDG 12 — Responsible Consumption and Production	Reduces wasted engineering hours and the compute waste caused by undetected or long-running incidents; local, compact models avoid unnecessary use of large hosted services.

7.5 Future Scope

- Deployment on a public cloud with managed PostgreSQL and monitoring.
- Multi-tenant support so that several teams can use separate incident libraries.
- Connectors for real company log sources and common log formats.
- Integration with ticketing and chat-ops tools so that explanations and corrections flow into existing workflows.
- Larger or fine-tuned open-weight models and re-ranking of retrieved incidents to improve explanation quality.
- Support for metrics and traces in addition to logs.

Chapter 8: Conclusion

Engineers responsible for production systems are overwhelmed by logs and alerts that say something is wrong but not what it means or whether it has been seen before, and triage depends on the memory of experienced individuals. This project proposes an AI-Powered Incident Intelligence Platform that automates this first step. Logs are ingested from synthetic or external sources, buffered in Redis, stored in PostgreSQL and analysed by two complementary detectors — explainable rules with Drain3 and an unsupervised Isolation Forest. Each anomaly is matched, by pgvector similarity search over sentence-transformer embeddings, with the most relevant past incident, and a locally hosted Llama 3.2 model turns this context into a plain-English explanation. Severity triage and Slack alerts bring the right incidents to the right people, while a feedback loop converts operator corrections into a growing incident memory.

The expected contribution is a reproducible, one-command, zero-licence-cost prototype together with a rigorous evaluation method that compares detectors using precision, recall and F1-score on labelled anomaly windows. The work shows that explainable, privacy-preserving and self-improving incident triage can be built from open-source components, and it provides a base for future work such as cloud deployment, multi-tenant support and integration with real company log sources. The main limitations — evaluation on mostly synthetic data, a single-node deployment and dependence on the size of the local model and the incident library — are acknowledged and define the next steps.

References

[1]P. Notaro, J. Cardoso, and M. Gerndt, "A survey of AIOps methods for failure management," ACM Trans. Intell. Syst. Technol., vol. 12, no. 6, 2021. [verify citation]

[2]B. Beyer, C. Jones, J. Petoff, and N. R. Murphy, Eds., Site Reliability Engineering: How Google Runs Production Systems. Sebastopol, CA, USA: O'Reilly Media, 2016.

[3]P. Lewis et al., "Retrieval-augmented generation for knowledge-intensive NLP tasks," in Advances in Neural Information Processing Systems (NeurIPS), vol. 33, 2020.

[4]M. Du, F. Li, G. Zheng, and V. Srikumar, "DeepLog: Anomaly detection and diagnosis from system logs through deep learning," in Proc. ACM SIGSAC Conf. Computer and Communications Security (CCS), 2017, pp. 1285–1298.

[5]W. Meng et al., "LogAnomaly: Unsupervised detection of sequential and quantitative anomalies in unstructured logs," in Proc. Int. Joint Conf. Artificial Intelligence (IJCAI), 2019, pp. 4739–4745. [verify citation]

[6]X. Zhang et al., "Robust log-based anomaly detection on unstable log data," in Proc. ACM ESEC/FSE, 2019, pp. 807–817. [verify citation]

[7]P. He, J. Zhu, Z. Zheng, and M. R. Lyu, "Drain: An online log parsing approach with fixed depth tree," in Proc. IEEE Int. Conf. Web Services (ICWS), 2017, pp. 33–40.

[8]S. He, J. Zhu, P. He, and M. R. Lyu, "Experience report: System log analysis for anomaly detection," in Proc. IEEE Int. Symp. Software Reliability Engineering (ISSRE), 2016, pp. 207–218.

[9]F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation forest," in Proc. IEEE Int. Conf. Data Mining (ICDM), 2008, pp. 413–422.

[10]N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in Proc. EMNLP-IJCNLP, 2019, pp. 3982–3992.

[11]W. Wang et al., "MiniLM: Deep self-attention distillation for task-agnostic compression of pre-trained transformers," in Advances in Neural Information Processing Systems (NeurIPS), vol. 33, 2020.

[12]V. Karpukhin et al., "Dense passage retrieval for open-domain question answering," in Proc. EMNLP, 2020, pp. 6769–6781.

[13]V.-H. Le and H. Zhang, "Log-based anomaly detection with deep learning: How far are we?," in Proc. ACM/IEEE Int. Conf. Software Engineering (ICSE), 2022. [verify citation]

[14]J. Zhu et al., "Loghub: A large collection of system log datasets towards automated log analytics," arXiv preprint, 2020. [verify citation]

[15]T. Ahmed et al., "Recommending root-cause and mitigation steps for cloud incidents using large language models," in Proc. ACM/IEEE ICSE, 2023. [verify citation]

[16]J. Pan, S. L. Wong, and Y. Yuan, "RAGLog: Log anomaly detection using retrieval augmented generation," arXiv preprint, 2023. [verify citation]

[17]S. Ghosh et al., "How to fight production incidents? An empirical study on a large-scale cloud service," in Proc. ACM Symp. Cloud Computing (SoCC), 2022. [verify citation]

[18]A. Grattafiori et al., "The Llama 3 herd of models," arXiv preprint, 2024. [verify citation]

[19]FastAPI Documentation. [Online]. Available: https://fastapi.tiangolo.com

[20]PostgreSQL Global Development Group, PostgreSQL 16 Documentation. [Online]. Available: https://www.postgresql.org/docs/16/

[21]pgvector: Open-source vector similarity search for PostgreSQL. [Online]. Available: https://github.com/pgvector/pgvector

[22]Redis Documentation, "Redis Streams." [Online]. Available: https://redis.io/docs/

[23]F. Pedregosa et al., "Scikit-learn: Machine learning in Python," J. Mach. Learn. Res., vol. 12, pp. 2825–2830, 2011.

[24]Ollama Documentation. [Online]. Available: https://ollama.com

[25]D. Merkel, "Docker: Lightweight Linux containers for consistent development and deployment," Linux Journal, vol. 2014, no. 239, 2014.

[26]Drain3: Streaming log template miner. [Online]. Available: https://github.com/logpai/Drain3 [verify citation]

[27]Slack API Documentation, "Incoming Webhooks." [Online]. Available: https://api.slack.com/messaging/webhooks

[28]React Documentation and Vite Documentation. [Online]. Available: https://react.dev and https://vite.dev

[29]United Nations, "Transforming our world: The 2030 Agenda for Sustainable Development," Resolution A/RES/70/1, 2015.

Appendices

Appendices A–F contain supporting diagrams, samples and configuration listings. Items marked illustrative are to be checked against the final implementation before submission.

Appendix A: Full-Page System Architecture Diagram

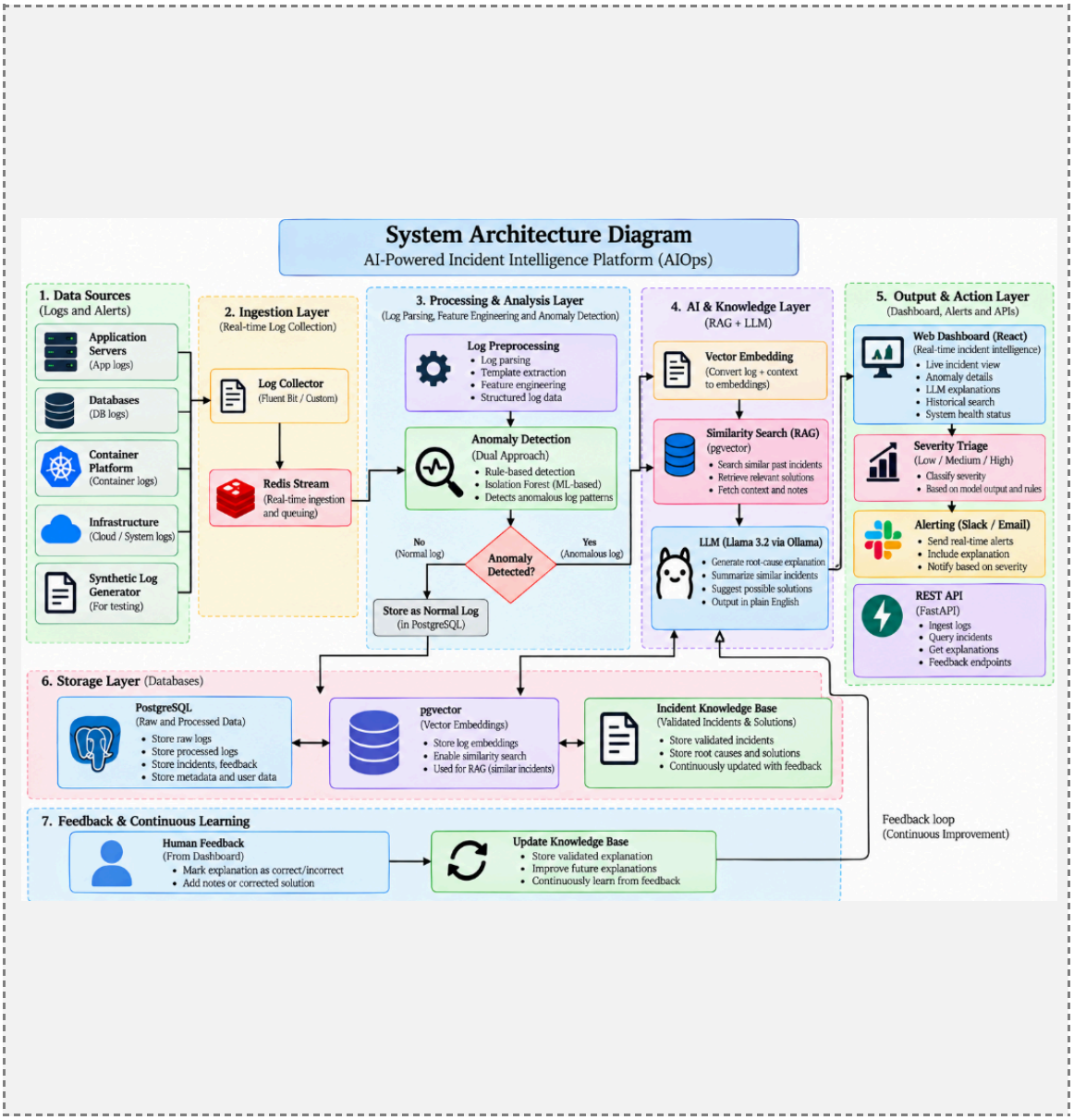


Figure 7: Full-page system architecture diagram

Appendix B: Database Schema Diagram

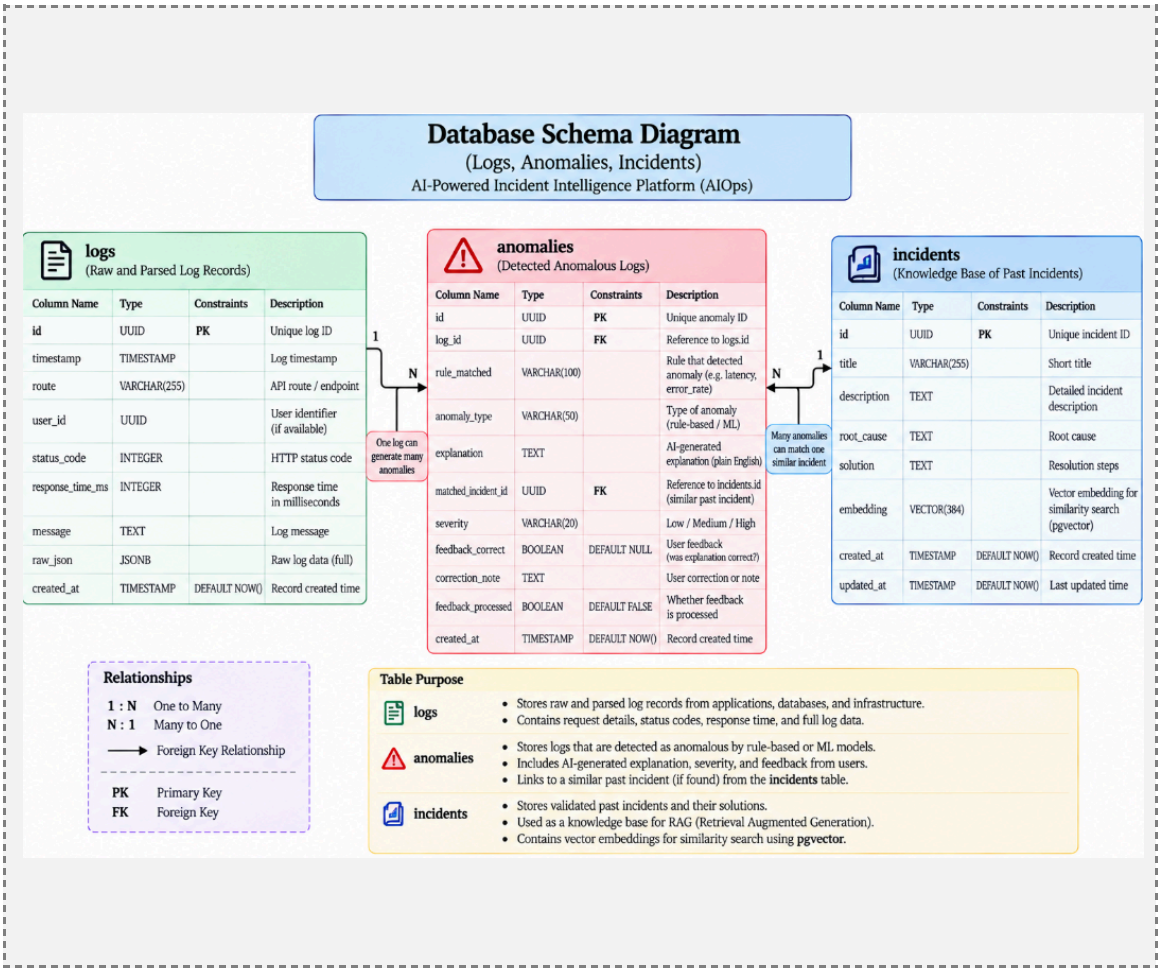


Figure 8: Database schema diagram (logs, anomalies, incidents)

Appendix C: Sample API Request and Response

The example below is illustrative; the header name and field names must be matched to the implemented API.

```
POST /ingest
X-API-Key: <API_KEY_PLACEHOLDER>
Content-Type: application/json

{
  "timestamp": "2026-01-01T10:15:30Z",
  "route": "/checkout",
  "user_id": "u_1024",
  "status_code": 500,
  "response_time_ms": 4200,
  "message": "Database connection pool exhausted"
}

--- Response (illustrative) ---
HTTP/1.1 200 OK
{ "status": "accepted" }
```

Table 19: REST endpoints exposed by the API service (illustrative)

Endpoint	Purpose
POST /ingest	Accept an external log record (API key required)
GET /stats	Summary counts for dashboard cards
GET /anomalies	Recent anomalies with explanation and matched incident
GET /logs	Recent log records
GET /health	Health indicator for each pipeline stage
POST /feedback	Operator marks an explanation correct or incorrect, with an optional note

Appendix D: Dashboard Screenshots

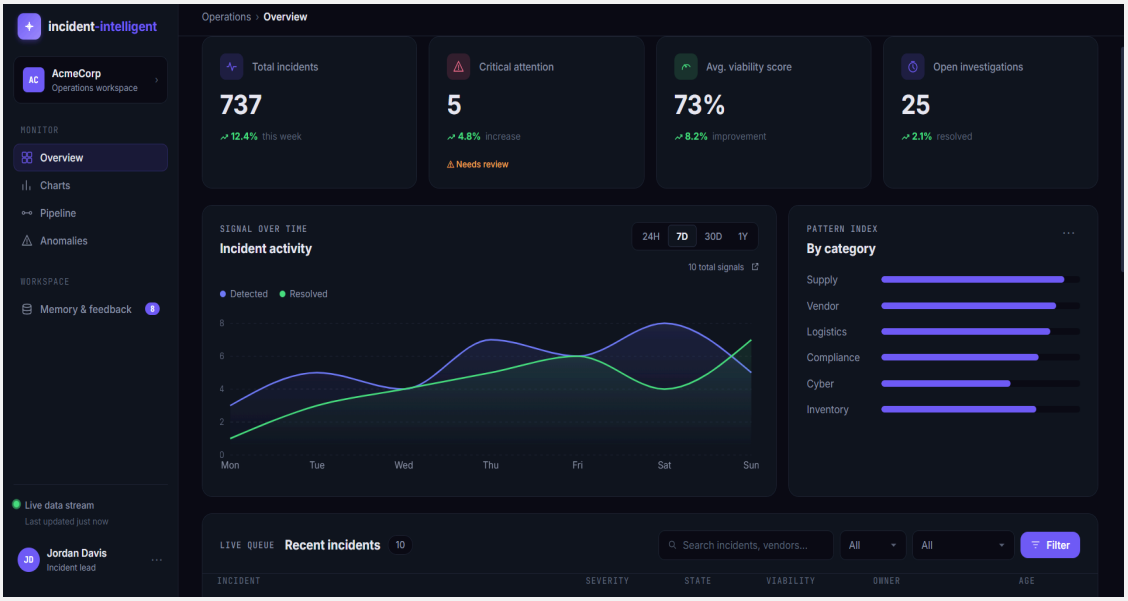


Figure 9: Dashboard overview



Figure 10: Anomalies-by-type chart

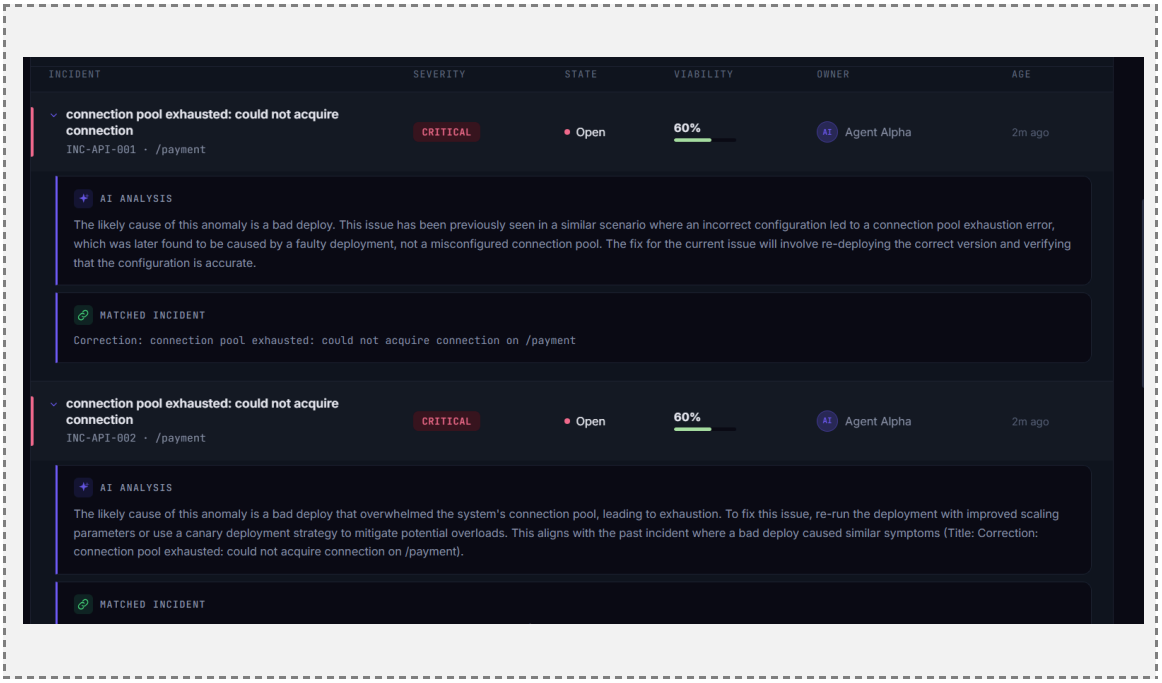


Figure 11: Expanded anomaly row with LLM explanation and matched past incident

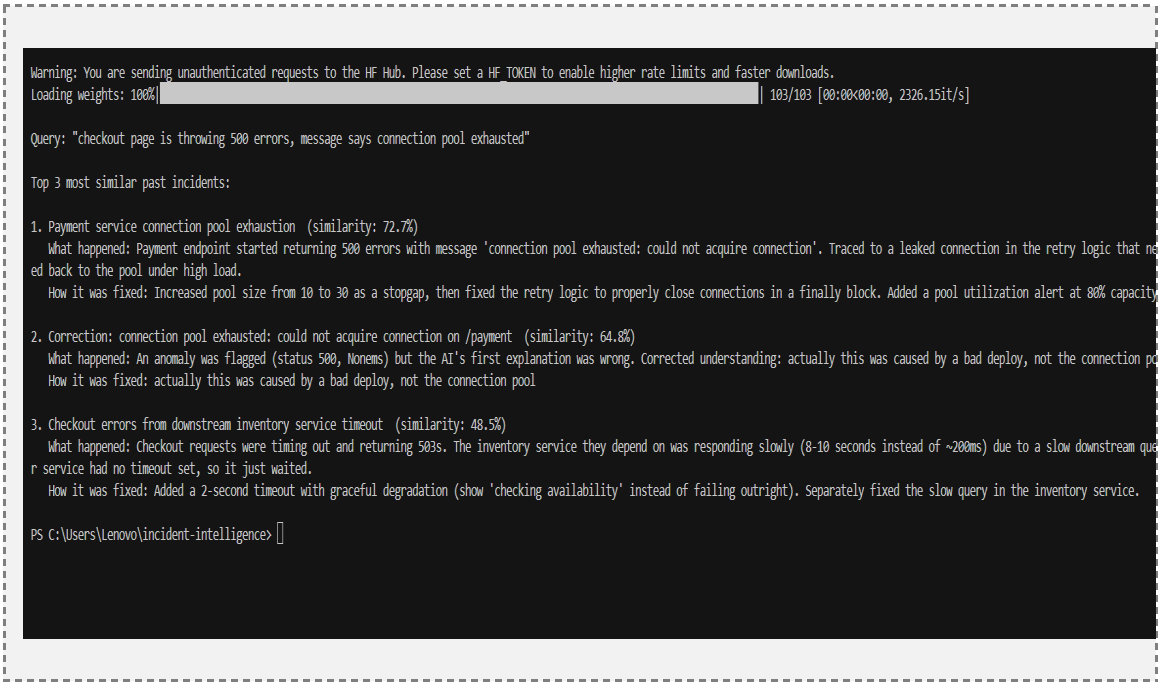


Figure 12: RAG system finding 3 similar past incidents



Figure 13: The evaluation compares three detectors using precision, recall, and F1-score

Appendix E: docker-compose Service List

Table 20: Services orchestrated by docker-compose

Service	Role	Notes
redis	Redis with the logs_stream stream	Buffering queue
postgres	PostgreSQL 16 with pgvector	logs, anomalies and incidents tables
consumer	Stream consumer	Persists stream records into logs
generator	Synthetic log generator	Produces traffic with labelled injected anomalies
rule analyzer	Rule-based detector with Drain3	Writes rule-matched anomalies
ml detector	Isolation Forest detector	Writes statistical-outlier anomalies
rag / llm explainer	Embedding, retrieval and Ollama explanation	Fills explanation and matched incident
triage / alerting	Severity rules and Slack webhook	Sets severity; sends alerts
feedback processor	Embeds corrections	Writes feedback into incidents
api	FastAPI service	REST endpoints
dashboard	React (Vite) application	Operator interface

The LLM runtime (Ollama) is accessed by the explainer service; [confirm whether Ollama runs as a compose service or on the host].

Appendix F: Sample .env Structure

All secrets are placeholders. Real values must never be committed to version control.

```
# Database
POSTGRES_USER=<user>
POSTGRES_PASSWORD=<password>
POSTGRES_DB=<database_name>
DATABASE_URL=postgresql://<user>:<password>@postgres:5432/<database_name>

# Queue
REDIS_URL=redis://redis:6379

# API security
INGEST_API_KEY=<API_KEY_PLACEHOLDER>

# LLM and embeddings
OLLAMA_HOST=<ollama_host_url>
LLM_MODEL=llama3.2
EMBEDDING_MODEL=all-MiniLM-L6-v2

# Alerting
SLACK_WEBHOOK_URL=<SLACK_WEBHOOK_PLACEHOLDER>
```